BugReel Pro

Physical Device Test Manual

A step-by-step, very-simple walkthrough for every test that needs a real phone

Test phone: Samsung Galaxy A05s

Version: 2026-10-11 · Covers Sprint 5 sign-off + Launch Readiness Gate

> **ℹ Why are we doing this?**
>
> Robots (automatic tests) already checked a lot of BugReel Pro. But some things only show up on a real phone held in a real hand: pop-up permission boxes, the floating bubble, Samsung's own battery rules, and the screen recorder. This little book walks you through each one, like following a recipe. You do not need to be a programmer. Just go slowly, do one step at a time, and tick the box at the end of each test.

How to use this book

Think of this book like building a LEGO set. Each page is one small piece. You do the pieces **in order**, and you don't skip ahead.

The four kinds of coloured boxes

> **✔ You should see**
>
> A **green box** tells you what the phone should look like after you do the steps. If what you see matches, the test is going well.

> **⚠ Uh-oh! If something looks wrong**
>
> An **orange box** tells you what to do if the phone does something different. Don't panic! Write down what happened and keep going with the next test unless the box says to stop.

> **i Why are we doing this?**
>
> A **blue box** explains why a test matters, so you know what you are protecting.

At the end of every test there is a little form with ☐ **PASS** and ☐ **FAIL**. Tick one. If you tick FAIL, write one sentence about what went wrong in the Notes line.

Words you will see

- **Tap** = touch the screen once with one finger and lift it.
- **Long-press** = touch and hold for about two seconds.
- **Swipe down from the top** = put your finger at the very top edge of the screen and slide it down. This opens the **notification shade** (the drawer of little messages).
- **Recents button** = the button with three little lines (|||) at the bottom left of the A05s. It shows all open apps.
- **Home button** = the middle button (□) at the bottom. It takes you to the home screen.
- **Back button** = the arrow (<) at the bottom right.
- **Computer** = the laptop or desktop that has the BugReel Pro code and Android Studio on it.

Golden rules

1. Do the parts in order: Part 1 → Part 2 → Part 3 → the Tests.
2. Only change one thing at a time.

3. If a test fails, take a screenshot (press **Power + Volume Down** at the same time) and write a note. Screenshots go into the Gallery app.
4. Keep the phone plugged into the computer during Parts 2 and 3.

Part 0 — What you need (your shopping list)

- The **Samsung Galaxy A05s**, charged to at least 70%.
- A **USB-C cable** that can carry data (not a charge-only cable). If the computer never notices the phone, try a different cable first.
- The **computer** with Android Studio and this project's code on it.
- A **Wi-Fi** connection for the phone.
- Access to the **Firebase console** (console.firebase.google.com) for the BugReel project.
- An **OpenAI API key** in `local.properties` (only needed for Test 7, AI Assist).
- This book and a pen.
- About **2 hours** for everything, plus a **7-day** "just use it" period at the end (Test 14).

> **ℹ Why are we doing this?**
>
> If any item is missing, a test later will get stuck. Collecting everything first is like putting all your ingredients on the table before you start baking.

Part 1 — Get the phone ready

1A. Turn on "Developer options" (the secret menu)

Phones have a hidden menu for people who build apps. We need to open it once.

1. On the phone, open **Settings** (the grey gear icon).
2. Scroll all the way down and tap **About phone**.
3. Tap **Software information**.
4. Find the line that says **Build number**. Tap it **7 times**, quickly, like knocking on a door.
5. If the phone asks for your PIN or pattern, enter it.

> **✓ You should see**
>
> A small message at the bottom: **"Developer mode has been turned on."**

1B. Let the computer talk to the phone (USB debugging)

1. Press **Back** twice to return to the main Settings list.
2. Scroll to the very bottom. There is now a new item called **Developer options**. Tap it.
3. Find **USB debugging** and turn the switch **on** (it turns blue). Tap **OK** on the warning.
4. Plug the phone into the computer with the USB-C cable.
5. A box pops up on the phone: **"Allow USB debugging?"**. Tick **"Always allow from this computer"** and tap **Allow**.

> **✓ You should see**
>
> In Android Studio, at the top of the window, the device dropdown now says **samsung SM-A057...** (that's the A05s's model code).

> **⚠ Uh-oh! If something looks wrong**
>
> No pop-up? Swipe down from the top of the phone. Tap the notification that says **"USB for file transfer"** or **"Charging this device via USB"**, and pick **Transferring files / Android Auto**.
>
> Still nothing? Try another cable, then another USB port on the computer.

1C. Write down the phone's software version

1. Settings → **About phone** → **Software information**.

2 Write down the **One UI version** and **Android version** here: One UI _____ Android _____

> **ℹ Why are we doing this?**
>
> Samsung changes how permissions look between versions. If someone else repeats this test later, they need to know which version you had.

Part 2 — Get the computer and Firebase ready

> **i Why are we doing this?**
>
> The latest code changes (security rules, sign-in, permission fixes) were written in a cloud computer that could **not** build Android apps. So the very first job is to let your computer build it and run the robot tests. If the robots fail, stop and fix that first. Testing a broken build on the phone wastes your time.

2A. Get the newest code

In a terminal (Command Prompt or PowerShell on Windows) inside the project folder, type:

```
git fetch origin
git checkout cl/loving-wright-i8p6q4
git pull
```

2B. Run the robot tests

```
.\gradlew test
.\gradlew lint
```

> **✔ You should see**
>
> Both commands finish with **BUILD SUCCESSFUL**.

> **⚠ Uh-oh! If something looks wrong**
>
> If you see **BUILD FAILED**, copy the red error lines and send them to the developer (or to Claude). Do not continue to the phone tests until this passes.

2C. Check your secret files are in place

- `local.properties` contains `OPENAI_API_KEY=sk-...`
- `app/google-services.json` exists. Download it from Firebase console → Project settings (gear icon) → Your apps → Android app → **google-services.json**.

2D. Switch on "Anonymous sign-in" in Firebase

BugReel signs each phone in quietly with an invisible guest account. Firebase must allow that.

1. Open **console.firebase.google.com** and pick the BugReel project.

2. In the left menu, tap **Build → Authentication**.
3. Open the **Sign-in method** tab.
4. Tap **Anonymous**, flip the switch to **Enable**, and press **Save**.

> **✔ You should see**
>
> Anonymous shows as **Enabled** in the list.

2E. Lock the doors (deploy the Security Rules)

Security rules are like a bouncer at a club door. They only let each tester see their own reports. In the terminal:

```
npm install -g firebase-tools
firebase login
firebase use --add (pick the BugReel project)
cd firebase-rules-tests
npm install
npm test (should say: pass 18, fail 0)
cd ..
firebase deploy --only firestore:rules,storage
```

> **✔ You should see**
>
> **Deploy complete!** In Firebase console → Firestore Database → **Rules** tab, the rules start with `rules_version = '2';` and mention `ownsExisting()`.

Part 3 — Put the app on the phone

3A. Start completely fresh

We want the phone to act like a brand-new user's phone. If BugReel Pro is already on it, remove it:

1. On the phone, long-press the **BugReel Pro** icon.
2. Tap **Uninstall**, then **OK**.

3B. Install the debug build

On the computer, with the phone plugged in:

```
.\gradlew installDebug
```

> **✔ You should see**
>
> **BUILD SUCCESSFUL** and **Installed on 1 device**. A **BugReel Pro** icon appears in the phone's app drawer (swipe up on the home screen to see all apps).

> **⚠ Uh-oh! If something looks wrong**
>
> "No connected devices"? Go back to Part 1B. Unplug and replug the cable, and check the "Allow USB debugging" box on the phone.

Test 1 — The "permission slip" (Consent screen)

i Why are we doing this?

The law says we must ask testers before we record anything, the same way a school asks parents to sign a permission slip before a trip. **This is a Launch Readiness Gate item. It must pass.**

1. Tap the **BugReel Pro** icon to open the app for the very first time.
2. Look at the screen. **Don't tap anything yet.**

✓ You should see

The title **"Before you start testing"**.

Text explaining that BugReel records the screen, screenshots and taps, backs up reports to your private cloud storage, and sends crash diagnostics.

One button: **"I agree, continue"**.

You **cannot** see the reports list or any capture button behind it.

3. Press the **Back** button (<).

✓ You should see

The app closes or goes to the background. It does **not** sneak you past the permission slip into the app.

4. Open BugReel Pro again. The consent screen should still be there.
5. Read the text out loud once. Does it make sense to a normal person? Write any confusing words in the Notes.
6. Tap **"I agree, continue"**.

✓ You should see

A new screen called **"Help improve BugReel's AI"**, with a checkbox **"Yes, use my reports to help train BugReel's AI"** and a **Continue** button.

Result for Test 1: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 2 — The "separate yes" (AI training opt-in)

> **i Why are we doing this?**
>
> Agreeing to use the app and agreeing to help train the AI are two **different** questions. It's like two separate stickers. Saying yes to one must not secretly say yes to the other.

1 On the "Help improve BugReel's AI" screen, look at the checkbox.

> **✔ You should see**
>
> The checkbox is **empty** (not ticked) to start with.

2 For this first run, **leave the box empty**, and tap **Continue**.

> **✔ You should see**
>
> The **Reports** screen appears with an empty-state message: **"Your first report will appear here. Start by recording a test session."** There is a **Start Capture** button, a **"Try Practice Mode first ->"** link, and an **Analytics** button at the top.

3 Swipe up from the bottom to go home, then open the **Recents** (|||) view and swipe BugReel Pro away to fully close it.

4 Open BugReel Pro again.

> **✔ You should see**
>
> You go **straight** to the Reports screen. The consent screens do not come back, because the phone remembered your answers.

Write it down: AI training opt-in for this run = **NO**. We'll test **YES** in Test 11.

Result for Test 2: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 3 — Practice Mode (the game)

> **i Why are we doing this?**
>
> Practice Mode should feel like a game: write, get a score straight away, try again. It only appears while you have zero reports, so we test it now, before making real reports.

1. On the Reports screen, tap **"Try Practice Mode first ->"**.
2. In the box **"Write your bug report attempt"**, type something lazy, like: app broke.
3. Tap **Score it**.

> **✔ You should see**
>
> A score like **"Score: 5/100"** (a low number) and some tips that start with a dot (•).

4. Now write a better attempt, e.g.: Login button does nothing. 1. Open app 2. Type email 3. Tap Login. Expected: home screen. Actual: nothing happens. Happens every time.
5. Tap **Score it** again.

> **✔ You should see**
>
> The score goes **up** compared to your lazy try.

6. Tap **Try another scenario**, then **Done practicing**.

> **✔ You should see**
>
> You are back on the Reports screen.

Result for Test 3: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 4 — Asking for phone permissions

ℹ Why are we doing this?

On the A05s, Android won't let an app show a notification or float a bubble over other apps until you say "yes". These were missing before and were added in the latest update, so watch closely.

1 On the Reports screen, tap **Start Capture**.

✔ You should see

A pop-up: **"Allow BugReel Pro to send you notifications?"**

Behind it, the capture screen says **"Capture bugs like a pro"**.

2 Tap **Allow**.

✔ You should see

A card called **"Floating capture button"** with a button **"Allow floating button"**.

3 Tap **Allow floating button**.

4 Samsung opens a settings page called **"Appear on top"** (this is Samsung's name for "display over other apps"). Find **BugReel Pro** and turn its switch **on**.

5 Press **Back** until you return to BugReel Pro.

✔ You should see

The "Floating capture button" card has **disappeared** by itself.

⚠ Uh-oh! If something looks wrong

If the card is still there after you return, write "overlay card did not refresh". Then close and reopen the app and see if it goes away.

If Samsung's page opened the full app list instead, look for BugReel Pro in that list.

Result for Test 4: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 5 — Recording a bug (the big one)

> **ℹ Why are we doing this?**
>
> This is the heart of the app: record the screen while the tester uses another app, then turn that recording into a report.

5A. Start recording

1 On the capture screen, tap **Start Capture**.

> **✔ You should see**
>
> Samsung shows a box like **"Start recording or casting with BugReel Pro?"**.
>
> On newer Samsung software there's a choice: **"A single app"** or **"Entire screen"**.

2 Choose **Entire screen** (if asked) and tap **Start** (or **Start now**).

> **✔ You should see**
>
> The BugReel screen says **"Recording in progress"** with a **Stop Capture** button.
>
> A small round **floating bubble** appears on the edge of the screen.
>
> Swipe down from the top: there's a BugReel Pro notification and Samsung's own red **screen-recording/casting icon** in the status bar.

5B. Use another app

3 Press **Home**. The bubble should stay floating on top of the home screen.

4 Open **Settings** (or any app) and tap around for about **20 seconds**. Pretend you are testing it.

5 Drag the bubble with your finger to a new spot. It should follow your finger.

5C. Stop recording

6 Press the **Recents** button (|||) and tap **BugReel Pro** to go back to it.

7 Tap **Stop Capture**.

> **✔ You should see**
>
> Briefly: **"Capture complete — building your report…"**
>
> Then the report builder opens, saying **"Step 1 of 5"**.
>
> The floating bubble disappears.

> **⚠ Uh-oh! If something looks wrong**
>
> Samsung asks for permission **every time** you start recording. That is normal on Samsung, and the app expects it.
>
> If you tap **Cancel** on Samsung's box, BugReel should say **"Screen recording permission was denied."** and go back to the Start Capture button. Try this once on purpose and tick it below.
>
> Tapping the bubble while recording does nothing in this version. That is expected; stop from inside the app.

Extra check: ☐ Cancelling Samsung's box showed "permission was denied" and did not crash.

Result for Test 5: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 6 — Keep recording when the screen is off

ℹ Why are we doing this?

Samsung phones try to save battery by stopping apps in the background. If they stop BugReel mid-recording, the tester loses their evidence.

1. Go back to the Reports screen (press Back if you're in the report builder; it's fine to lose that draft) and start a new capture like in Test 5A.
2. Press **Home**, then press the **Power** button to lock the phone.
3. Wait **2 full minutes** (use a clock).
4. Unlock the phone, open BugReel Pro from Recents, and tap **Stop Capture**.

✔ You should see

The report builder opens normally ("Step 1 of 5"), and the app did not crash or forget the recording.

⚠ Uh-oh! If something looks wrong

If recording stopped by itself: open **Settings → Apps → BugReel Pro → Battery** and pick **Unrestricted**. Then repeat this test and write down that you needed this.

That note matters: it tells the team whether to add Samsung to the "needs battery setup" list in the code.

Result for Test 6: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 7 — Building a report (5 small steps)

> **i Why are we doing this?**
>
> A good report is the product. This test walks every step of the report builder and the AI helper.

Start a capture and stop it (Test 5) so you are on **"Step 1 of 5"**.

Step 1 of 5 — Title and description

1. Tap **Title** and type: Settings search crashes on emoji
2. Tap **Description** and type at least two sentences about the pretend bug (more than 50 letters).
3. Pick a **Platform** from the dropdown (e.g. UTEST). Tap **Next**.

Step 2 of 5 — Severity and frequency

4. Pick a **Severity** (e.g. MEDIUM) and a **Frequency** (e.g. ALWAYS). Tap **Next**.

Step 3 of 5 — Repro steps (how to make the bug happen)

5. In **Add step**, type Open Settings and tap **Add**.
6. Add two more: Tap the search icon and Type an emoji.
7. Try the little ↑ and ↓ arrows to move a step, and × to delete one, then add it back. Tap **Next**.

Step 4 of 5 — Evidence

8. Tap **Add Evidence**. The "Attach Evidence" screen opens.
9. Tap **Add Screenshot** and pick the screenshot you took earlier from the Gallery.
10. Optionally paste some text into **Paste log text**. Tap **Done**, then **Next**.

> **✔ You should see**
>
> The evidence summary shows **Recording: attached** (from your capture) and **Screenshots: 1**.

Step 5 of 5 — Review and AI Assist

11. Tap **AI Assist**. Wait a few seconds (this calls OpenAI over the internet).

✔ You should see

AI suggestions for title, summary and **Repro steps** appear, **coloured or labelled differently** from your own text so you can tell them apart.

OR the message **"AI suggestion was blocked — showing your original text"**. That is the safety checker doing its job, and still counts as a pass.

12 Change one word in the AI's title (so we record that you edited it). Tap **Apply**.

13 Back on the review step, tap **Submit**.

✔ You should see

The **Quality Score** screen, with a big number counting up (animated) to the score.

Five rows: Reproduction, Evidence, Description, Severity, Metadata.

If the score is below 70: a message about scores below 70 being less likely to be accepted, plus tips.

Buttons **Report Outcome** and **Done**.

⚠ Uh-oh! If something looks wrong

Turn on **Airplane mode** and try AI Assist once more on a new report. The app must **not** freeze or go blank. It should keep your own text.

Tap AI Assist many times fast (more than 20 in a minute). You should see a friendly countdown **"You've reached the AI limit. Try again in X seconds."**, never a silent failure.

Result for Test 7: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 8 — Telling BugReel what the platform decided

i Why are we doing this?

This is **the flywheel**, the company's most important data. When uTest/Testlio accepts or rejects a report, the tester tells BugReel. Over time, that teaches BugReel what good reports look like.

1 On the Quality Score screen, tap **Report Outcome**.

✔ You should see

A screen asking **"What did the platform decide?"** with two round buttons: ACCEPTED and REJECTED.

1 Tap **REJECTED**.

✔ You should see

A new box appears: **"Why was it rejected?"**

3 Type: Duplicate of an existing bug. Tap **Submit**.

4 Make a second report (repeat Test 5 + Test 7 quickly) and this time mark it **ACCEPTED**.

Result for Test 8: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 9 — Peek inside the phone's notebook (database)

> **i Why are we doing this?**
>
> The Sprint 5 stop condition says: **"OutcomeFeedbackDao has non-zero rows from internal testing."** In plain words: did the phone actually write down the outcomes from Test 8?

1. Keep the phone plugged in and BugReel Pro **open on screen**.
2. In Android Studio, open **View → Tool Windows → App Inspection**.
3. At the top of that panel, choose the A05s and the process **com.bugreel.pro**.
4. Click the **Database Inspector** tab, then open **bugreel.db → outcome_feedback**.

> **✔ You should see**
>
> At least **2 rows**, one for each report you submitted.
>
> One row with platformOutcome = **REJECTED** and rejectionReason = Duplicate of an existing bug; one with **ACCEPTED**.
>
> preAiQualityScore and postAiQualityScore have numbers in them (not both 0 on the report where you used AI Assist).
>
> testerOverrodeSuggestion = **1** (true) on the report where you changed the AI's title.

Write the number of rows here: _____

> **⚠ Uh-oh! If something looks wrong**
>
> If the table is empty, this is a **launch blocker**. Take a screenshot of the Database Inspector and mark FAIL.

Result for Test 9: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 10 — The Analytics dashboard

1 Go back to the Reports screen (tap **Done** on the Quality Score screen).

> **✔ You should see**
>
> Your reports are listed. Tapping one opens its Quality Score screen again.

2 Tap **Analytics** at the top right.

> **✔ You should see**
>
> An **Acceptance rate** card (with 1 accepted and 1 rejected, that's **50%**).
>
> A **Quality trend** section showing your scores.

Bonus empty-state check: after Test 13 (fresh install) open Analytics with zero reports. It should show a faded preview of what the chart will look like, not just "No data".

Result for Test 10: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 11 — Does the cloud get the right things?

> **ℹ Why are we doing this?**
>
> Reports are backed up to Firebase. But outcome data (the AI-training data) must **only** be uploaded if the tester said **YES** in the AI opt-in. In Test 2 you said NO.

11A. With opt-in = NO (your current setup)

1. Make sure the phone has Wi-Fi. Wait about **2 minutes**.
2. Open Firebase console → **Firestore Database** → **Data** tab.

> **✔ You should see**
>
> A **reports** collection with your reports. Each document has a **userId** that is a long random code (NOT the word "anonymous") and a **data** field.
>
> **No outcome_feedback** documents from this phone.

11B. With opt-in = YES

3. On the phone: **Settings → Apps → BugReel Pro → Storage → Clear data**. (This makes the app forget everything, like a brand-new install.)
4. Open BugReel Pro. Agree to consent, and this time **tick** the AI training box, then Continue.
5. Make one report and mark its outcome as ACCEPTED (Tests 5, 7, 8).
6. Wait 2 minutes, then refresh the Firestore page.

> **✔ You should see**
>
> An **outcome_feedback** collection now exists. The document's name is the report's ID, and its userId matches the new report's userId.

11C. Offline, then online

7. Turn on **Airplane mode**. Make and submit one more report.
8. Wait 1 minute. Check Firestore: the new report is **not** there yet (correct, you're offline).
9. Turn Airplane mode **off**. Wait 2 minutes and refresh.

> **✔ You should see**
>
> The new report appears in Firestore by itself.

Result for Test 11: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 12 — Crash reports only after consent

ℹ Why are we doing this?

Crashlytics tells the team when the app crashes. But it should stay quiet until the tester agrees on the consent screen.

1. In Firebase console, open **Release & Monitor → Crashlytics**.
2. Check that the app appears and is receiving data. It can take up to 24 hours after the first launch to show up.

✔ You should see

Crashlytics shows the app and a **crash-free users/sessions** number.

Note: this test can be finished the next day. Write the crash-free % you see: _____ %

Result for Test 12: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 13 — The "real shop" version (release build)

> **i Why are we doing this?**
>
> The version testers get from Google Play is squeezed and scrambled ("minified") to make it smaller. Sometimes squeezing breaks things that worked in the normal version, so we test it separately.

1. Ask the developer to add the signing keys to `local.properties` (KEYSTORE_PATH, KEYSTORE_PASSWORD, KEY_ALIAS, KEY_PASSWORD).
2. Uninstall BugReel Pro from the phone.
3. On the computer run:

```
.\gradlew assembleRelease
adb install app\build\outputs\apk\release\app-release.apk
```

4. Quickly repeat: **Test 1** (consent), **Test 5** (record), **Test 7** (build + submit, with AI Assist), **Test 8** (outcome), **Test 10** (analytics).

> **✔ You should see**
>
> Everything works exactly like the debug version. No sudden closing of the app.

> **⚠ Uh-oh! If something looks wrong**
>
> If the release app closes by itself ("BugReel Pro keeps stopping"), plug in the phone and run `adb logcat -d > crash.txt`, then send crash.txt to the developer. It usually means a missing "keep" rule in proguard-rules.pro.

Result for Test 13: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

Test 14 — One week of normal use

> **i Why are we doing this?**
>
> The Launch Readiness Gate needs **7 days** and **at least 100 sessions** (app openings) with more than **99.5% crash-free**. This is just using the app like a real tester would.

1. Install the **release** build (Test 13) on the A05s, plus any other team phones.
2. Each day for 7 days, open BugReel Pro several times. Record at least one real bug a day and enter its real outcome when the platform replies.
3. Write any problem in a list with the date, what you did, and what happened.
4. On day 7, check Crashlytics (Test 12) and the outcome_feedback count (Test 9).

Day	Sessions	Reports	Crashes?	Notes
1				
2				
3				
4				
5				
6				
7				

Result for Test 14: ☐ PASS ☐ FAIL

Notes: _____

Date: _____ Tester initials: _____

When something goes wrong: a quick helper

- **The app closed by itself.** Plug in, run `adb logcat -d > crash.txt` right away, and keep the file.
- **A button does nothing.** Wait 5 seconds. Still nothing? Screenshot, note which button, move on.
- **Samsung keeps asking to allow recording.** Normal for Samsung. Not a bug.
- **The floating bubble never appears.** Check Settings → Apps → BugReel Pro → **Appear on top** is ON. If it is ON and there's still no bubble, mark Test 5 FAIL.
- **No notification.** Check Settings → Apps → BugReel Pro → **Notifications** is ON.
- **Nothing reaches Firebase.** Check Anonymous sign-in is enabled (Part 2D) and the phone has internet. Then wait a few minutes. Uploads happen in the background, not instantly.
- **To start over completely:** Settings → Apps → BugReel Pro → Storage → **Clear data**.

Final sign-off sheet

When every test above is ticked, fill this in. It is what CLAUDE.md calls the "human device-test sign-off".

#	Test	PASS / FAIL
1	Consent screen (permission slip)	
2	Separate AI training opt-in	
3	Practice Mode	
4	Notification + Appear-on-top permissions	
5	Screen recording + floating bubble	
6	Recording survives screen-off	
7	Report builder + AI Assist + Quality Score	
8	Outcome entry (flywheel)	
9	outcome_feedback rows in database (non-zero)	
10	Analytics dashboard	
11	Cloud sync + opt-in respected	
12	Crashlytics receiving after consent	
13	Release (minified) build smoke test	
14	7-day internal use, >99.5% crash-free	

Phone: Samsung Galaxy A05s · One UI _____ · Android _____

Tester name: _____ Signature: _____ Date: _____

> **Still needed outside this book (not phone tests)**
>
> Legal review of the consent screen wording.
>
> Google Play Closed Testing submission, plus the Play review result for the floating bubble. If it comes back RED, set `BUBBLE_FORCE_FALLBACK` to true and repeat Tests 4–6.
>
> No open P0/P1 bugs from the notes in this book.